

PP_T2_2024W

⚠ Disclaimer

Alles kann falsch sein! Bitte bei Korrekturvorschlag melden: [discord](#)

A und B seien definiert durch `interface A<T> {}` und `class B<T> implements A<T> {}`.
 Bitte markieren Sie jedes Auswahlfeld, bei dem der links stehende Typ ein Untertyp des darüber stehenden Typs ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Typ (Links)	A<String>	A<? super String>	A<? extends String>	A<? >	Erklärung
A<Object>		x		x	Object is a supertype of String.
B<Object>		x		x	$B < Object > <: A < Object >$. Same as above.
A<String>	x	x	x	x	String is equal to, super c and extends String.
B<String>	x	x	x	x	$B < String > <: A < String >$. Same as above.
B<?>				x	Wildcard only fits unbound wildcard.
B<? super Integer>				x	Integer is unrelated to String.
B<? super String>		x		x	Inherits bounds from definit
B<? extends String>			x	x	Inherits bounds from definit
A<B<String>>				x	Complex type fits unbound wildcard.
B<A<String>>				x	Complex type fits unbound wildcard.

Aufgabe 2 (10 Punkte)

Jede Zeile enthält eine Java-Methode, die den Parameter mit einem anderen deklarierten Typ zurückgibt. Set und HashSet seien aus `java.util` importiert. Bitte markieren Sie das Auswahlfeld „statisch“ wenn bereits der Compiler ein Problem im Zusammenhang mit der Typänderung meldet (Syntaxfehler oder „unchecked“-Warnung), sonst „dynamisch“ wenn das Laufzeitsystem bei Ausführung der Methode eine Ausnahme wegen der Typänderung auslösen kann, sonst „fehlerfrei“.

Methode	Typ	Erklärung
<code>Object f(String x) {return x;}</code>	Fehlerfrei	Safe Upcast.
<code>String f(Object x) {return (String)x;}</code>	Dynamisch	Explicit Downcast. Runtime Check failure possible.
<code><T> HashSet<T> f(Set<T> x) {return (HashSet<T>)x;}</code>	Statisch	Unchecked cast warning (Type Erasure).
<code><T> T f(Object x) {return (T)x;}</code>	Statisch	Unchecked cast warning (Type Erasure).
<code><T> Set f(HashSet<T> x) {return x;}</code>	Fehlerfrei	Raw type assignment is valid (legacy support).
<code><T> HashSet<T> f(Set x) {return (HashSet<T>)x;}</code>	Statisch	Unchecked cast warning (Raw to Generic).
<code><T> Set<Set<T>> f(Set<HashSet<T>> x) {return x;}</code>	Statisch	Compiler Error: Generics are invariant on the type parameter level.
<code><T> Set<Set<T>> f(HashSet<Set<T>> x) {return x;}</code>	Fehlerfrei	Valid because outer type <code>HashSet</code> <code><: Set</code> and inner types match exactly.
<code><T> Set<Object[]> f(Set<T[]> x) {return x;}</code>	Statisch	Compiler Error: Generics are invariant. <code>Set<A></code> is not <code>Set</code> even if <code>A <: B</code> .
<code>Set[] f(HashSet[] x) {return x;}</code>	Fehlerfrei	Arrays are covariant (<code>HashSet[] <: Set[]</code>).

Aufgabe 3 (10 Punkte)

Bitte markieren Sie in jeder Zeile das eine Auswahlfeld, bei dem die links stehende Aussage am ehesten eine Eigenschaft der darüber stehenden Parametrisierungsform in Java oder AspectJ ist.

Annotationen Aspekte Generizität

Aussage	Annotationen	Aspekte	Generizität	Erklärung
wird auch <i>parametrischer Polymorphismus</i> genannt			X	Generics erlauben Typen als Parameter (z.B. <code><T></code>), das ist die Definition dieses Polymorphismus.
<code>before()</code> -Advices werden vor normalem Code ausgeführt		X		Das Schlüsselwort <code>before()</code> gehört zur Syntax von AspectJ (Advices).
Information steht der gesamten Werkzeugkette zur Verfügung	X			Annotationen können vom Compiler, Tools (Javadoc) und der Runtime gelesen werden.
zahlreiche Typen von <i>Pointcuts</i> werden unterschieden		X		Pointcuts (Schnittstellen im Codefluss) sind das Herzstück der Aspektorientierung.
binäre Methoden sind über rekursive Deklarationen ausdrückbar			X	Bezieht sich auf F-Bounded Polymorphism (z.B. <code>Comparable<T></code>), wo ein Typ relativ zu sich selbst definiert wird.
<code>..</code> steht für eine beliebige Anzahl jedes beliebigen Zeichens		X		Wildcard-Syntax in AspectJ Pointcuts (z.B. für Argumente oder Package-Namen).
fast ausschließlich für <i>Querschnittsfunktionalität</i> geeignet		X		AOP wurde erfunden, um "Cross-Cutting Concerns" (Logging, Security) zu kapseln.
mit <code>@Target</code> wird festgelegt, wo Information anheftbar ist	X			<code>@Target</code> ist eine Meta-Annotation in Java.
die Syntax ist an die zur Definition von Interfaces angelehnt	X			Die Definition lautet <code>public @interface Name {}</code> .

Aussage	Annotationen	Aspekte	Generizität	Erklärung
zur Laufzeit sind Daten über Objekte von <code>Class</code> zugreifbar	X			Via Reflection (<code>getAnnotation()</code>), wenn die RetentionPolicy <code>RUNTIME</code> ist.

Aufgabe 4 (10 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem der links stehende Typausdruck (mit Typnamen aus den Paketen `java.util.function` und `java.lang`) ein Typ des darüber stehenden Lambdas ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Typ \ Lambda	<code>Long::max</code>	<code>(x,y) -> x-y</code>	<code>x -> y -> x*y</code>	<code>x -> x*x</code>	<code>() -> 0</code>	
<code>BiFunction<Long,Long,Long></code>	☒	☒				
<code>BiFunction<Long,Long,Integer></code>						<i>Alle falsch:</i> Rückgabotyp ist <code>Long</code> , nicht <code>Integer</code> .
<code>BiFunction<Integer,Long,Long></code>						<i>Alle falsch:</i> Erster Parameter muss <code>Long</code> sein.
<code>LongUnaryOperator</code>				☒		<code>Long -> Long</code> (Parameter).
<code>Function<Long,LongUnaryOperator></code>			☒			<code>Long -> (Long -> Long)</code> .
<code>Function<Long,Function<Long,Long>></code>			☒			Identisch zur Zeile darüber (Generics vs. spezialisierter Typ).
<code>BinaryOperator<Long></code>	☒	☒				Spezialisierung von <code>BiFunction</code> (alle Typen gleich).
<code>LongBinaryOperator</code>	☒	☒				Primitive Variante von <code>BinaryOperator</code> .
<code>IntSupplier</code>					☒	<code>() -> int</code> (0 liefert int).
<code>LongSupplier</code>					☒	<code>() -> Long</code> (0 passt auch in long).

Aufgabe 5 (10 Punkte)

- Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Eigenschaft auf den darüber stehenden Methodenaufruf zutrifft. Es können **keines, eines oder mehrere** Felder pro Zeile auszuwählen sein.

Eigenschaft	<code>wait()</code>	<code>notifyAll()</code>	Thread <code>.sleep(5)</code>	Erklärung
weckt alle im Synchronisationsobjekt wartenden Threads auf		x		<code>notifyAll</code> weckt alle Threads an <i>diesem</i> Monitor warten.
weckt alle im gesamten System wartenden Threads auf				<i>Alle falsch:</i> Es werden nur Threads geweckt, die auf <i>genau diesem</i> Monitor warten, nicht alle im OS.
Verwendung in einem <code>synchronized</code> -Block ist gefährlich			x	<i>Wait:</i> Sicher (gibt Lock frei). <i>Sleep:</i> Gefährlich, da der Thread den Lock <i>behält</i> und alles blockiert.
ist nur innerhalb eines <code>synchronized</code> -Blocks verwendbar	x	x		Sonst fliegt zur Laufzeit eine <code>IllegalMonitorStateException</code> .
gehört zu den grundlegenden Synchronisationsmechanismen	x	x		Methoden von <code>java.lang.Object</code> .
kann eine <code>InterruptedException</code> zurückgeben	x		x	<code>wait</code> und <code>sleep</code> können unterbrochen werden.
suspendiert die Ausführung des aktuellen Threads	x		x	Beide pausieren den Thread (einer wartet auf Signal, der andere auf Zeit).
suspendiert die Ausführung auf unbestimmte Zeit	x			<code>sleep(5)</code> ist zeitlich begrenzt, <code>wait()</code> wartet ewig auf ein <code>notify</code> .
bei Rückkehr ist der Lock auf den aktuellen Thread gesetzt	x			Nach dem Aufwachen aus <code>wait</code> muss der Thread den Lock erst wiedererlangen.
gibt Lock des Synchronisationsobjekts vorübergehend frei	x			Der entscheidende Unterschied: <code>sleep</code> : <code>wait</code> lässt andere Threads rein.

Aufgabe 6 (10 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Kommandozeile (in **bash** ausgeführt) die darüber stehende Auswirkung hat. Es können **keines**, **eines** oder **mehrere** Felder pro Zeile auszuwählen sein.

Legende für die Spaltenüberschriften

- **gleichz.:** mehrere Prozesse laufen gleichzeitig (Parallelität)
- **Pipeline:** Prozesse über Pipeline verbunden (`|` oder `|&`)
- **Hintergr.:** Prozesse laufen im Hintergrund (`&` am Ende oder in Subshell)
- **cat <:** Standardeingabe von `cat` umgeleitet (Input Redirection)
- **cat >:** Standardausgabe von `cat` umgeleitet (Output Redirection)
- **cat 2>:** Fehlerausgabe von `cat` umgeleitet (Error Redirection)

Befehl	gleichz.	Pipeline	Hintergr.	cat <	cat >	cat 2>
<code>cat < a wc > b</code>	x	x		x	x	
<code>cat a & wc &>> b</code>	x	x			x	x
<code>cat < a & wc &> b &</code>	x	x	x	x	x	x
<code>cat a wc b</code>						
<code>for i in *; do (cat i >.. /bak/\$i &); done</code>	x		x		x	
<code>for i in *; do cat i wc > ../wc/\$i; done</code>	x	x			x	
<code>if test cat a = "a b c"; then wc b; else wc c; fi</code>						
<code>cat a; wc a</code>						
<code>cat a &> b && wc b</code>					x	x
<code>cat wc</code>	x	x			x	

Wohin fließen die Daten?

- `cmd > file`: Leitet **Stdout** in Datei um (überschreibt).
- `cmd >> file`: Leitet **Stdout** in Datei um (hängt an).
- `cmd < file`: Liest **Stdin** aus Datei (statt Tastatur).
- `cmd 2> file`: Leitet nur **Stderr** (Fehler) um.
- `cmd &> file`: Leitet **beides** (Stdout + Stderr) in Datei um.

Verbindet Ausgabe von A mit Eingabe von B. **Laufen immer gleichzeitig (parallel)!**

- `A | B`: Leitet nur **Stdout** von A an B weiter. Fehler landen am Bildschirm.
- `A |& B`: Leitet **Stdout UND Stderr** von A an B weiter.

Wie und wann laufen Befehle?

- `A ; B`: **Sequenz**. Erst A fertigmachen, dann B (egal ob A geklappt hat).

- `A && B`: **UND**. B läuft nur, wenn A **erfolgreich** war (Exit Code 0).
- `A || B`: **ODER**. B läuft nur, wenn A **fehlgeschlagen** ist (Exit Code != 0).
- `A &`: **Hintergrund**. A startet, Shell wartet nicht, User kann sofort weitertippen.

Aufgabe 7 (10 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Aussage eine Eigenschaft des darüber stehenden Entwurfsmusters ist. Es können keines, eines oder mehrere Felder pro Zeile auszuwählen sein.

Eigenschaft	Decorator	Proxy	Iterator	Prototype	Factory-Method	Erklärung
führt zu vielen kleinen Objekten	x					Jede "Dekoration" ist eine neue Objekt-Instanz (Wrapper).
unterstützt Umkehrung der Abhängigkeiten					x	<i>Factory Method</i> : Der Client hängt von Abstraktion ab, nicht von konkreter Klasse (DIP).
ist erzeugendes Entwurfsmuster				x	x	
ist Entwurfsmuster für Struktur	x	x				Decorator/Proxy bauen Strukturen.
hilft große Zahl an Klassen zu vermeiden	x			x		<i>Decorator</i> verhindert Unterklassen-Explosion. <i>Prototype</i> spart Factory-Hierarchien.
häufig als innere Klasse implementiert			x			Meistens <code>private class Itr implements Iterator</code> .
Objektidentität ist damit unzuverlässig	x					Man hat Referenz auf den Wrapper, nicht das Original (<code>==</code> schlägt fehl).
beruht auf Delegation	x	x				Leiten Aufrufe an das gekapselte Objekt weiter.
oft große Anzahl an Unterklassen nötig					x	<i>Factory Method</i> : Parallel zur Produkthierarchie braucht man oft Creator-Hierarchie.

Eigenschaft	Decorator	Proxy	Iterator	Prototype	Factory-Method	Erklärung
für oberflächliche Erweiterungen geeignet	x					Man kann zur Laufzeit Verantwortung hinzufügen.

Aufgabe 8 (10 Punkte)

Bitte markieren Sie in jeder Zeile das eine Auswahlfeld, bei dem die links stehende Aussage am ehesten eine Eigenschaft des darüber stehenden Entwurfsmusters ist.

Aussage	Decorator	Proxy	Iterator	Prototype	Factory-Method	Erklärung
kann mit kovarianten Problemen umgehen					x	In Java können überschriebene Methoden in Subklassen einen spezifischeren Rückgabotyp haben (Kovarianz). Die Factory-Methode nutzt das, um konkrete Produkte zurückzugeben.
es gibt externe und interne Varianten			x			Extern: Der Client kontrolliert den Fluss (z.B. <code>next()</code>). Intern: Der Iterator führt eine Operation auf allen Elementen aus (z.B. <code>forEach()</code>).
mehrere gleichzeitige Abarbeitungen möglich			x			Da jeder Iterator seinen eigenen Zeiger (Cursor) speichert, können mehrere Iteratoren gleichzeitig und unabhängig dieselbe Liste durchlaufen.
zyklische Strukturen bereiten Probleme				x		Beim "Deep Copy" (tiefen Klonen) führen zirkuläre Referenzen (A kennt B, B kennt A) ohne Spezialbehandlung zu Endlosschleifen.

Aussage	Decorator	Proxy	Iterator	Prototype	Factory-Method	Erklärung
Verantwortlichkeiten wieder entziehbar	x					Dekoratoren werden zur Laufzeit geschachtelt. Man kann die Dekoration entfernen oder die Kette neu aufbauen (im Gegensatz zur statischen Vererbung).
robuste Varianten werden bevorzugt			x			Ein "robuster" Iterator erlaubt es die zugrundeliegende Collection während der Iteration zu verändern (z.B. Elemente zu löschen), ohne abzustürzen.
flache von tiefen Kopien unterschieden				x		Das ist die zentrale Unterscheidung beim Klonen: Kopiert man nur die Referenzen (flach) oder die ganzen Objekte rekursiv (tief)?
führt zu parallelen Klassenhierarchien					x	Ein klassischer Nachteil: Für fast jede neue Produktklasse muss oft auch eine neue Creator-Klasse (Factory) erstellt werden.
schlecht geeignet für umfangreiche Objekte				x		da das Klonen riesiger Speicherstrukturen teuer ist.

Aussage	Decorator	Proxy	Iterator	Prototype	Factory-Method	Erklärung
kein Proxy, aber gleiche Struktur möglich	x					Decorator und Proxy haben fast identische UML-Strukturen (Interface + Referenz auf Objekt), unterscheiden sich aber in der Absicht (Funktion vs. Zugriffskontrolle).

Aufgabe 9 (10 Punkte)

Bitte markieren Sie in jeder Zeile das eine Auswahlfeld, bei dem die links stehende Aussage am ehesten eine Eigenschaft des darüber stehenden Entwurfsmusters ist.

Aussage	Auswahl	Erklärung
wird auch <i>Cursor</i> genannt	Iterator	GoF (Gang of Four) listet "Cursor" explizit als Synonym für den Iterator, da er sich wie ein Zeiger über eine Menge bewegt.
wird auch <i>Wrapper</i> genannt	Decorator	Ein Decorator umschließt ("wraps") eine Komponente, um ihr responsibilities hinzuzufügen.
wird auch <i>Surrogate</i> genannt	Proxy	Proxy ist per Definition ein Stellvertreter (<i>Surrogate</i>) oder Platzhalter, um den Zugriff auf ein Objekt zu kontrollieren.
wird auch <i>Virtual-Constructor</i> genannt	Factory-Method	Erlaubt Subklassen zu entscheiden, welche Klasse instanziiert wird. Da dies zur Laufzeit dynamisch wirkt, nennt man es virtuellen Konstruktor.
<i>Smart-Reference</i> ist eine Variante davon	Proxy	Ein <i>Smart Reference Proxy</i> ist eine spezifische Implementierung, die zusätzliche Aktionen (z.B. Reference Counting) beim Zugriff auf das Objekt durchführt.
<i>Subject</i> ist ein Bestandteil	Proxy	Das Pattern definiert ein gemeinsames Interface <code>Subject</code> , das sowohl vom <code>RealSubject</code> als auch vom <code>Proxy</code> implementiert wird.
<i>Aggregate</i> ist ein Bestandteil	Iterator	Das <code>Aggregate</code> (oder Container) ist das Interface, das die Methode <code>CreateIterator()</code> definiert.
<i>Creator</i> ist ein Bestandteil	Factory-Method	Der <code>Creator</code> ist die Klasse, die die Factory-Methode deklariert und oft eine Default-Implementierung bereitstellt.
<i>Product</i> ist ein Bestandteil	Factory-Method	<code>Product</code> definiert das Interface der Objekte, die die Factory-Methode erzeugt.
<i>Component</i> ist ein Bestandteil	Decorator	<code>Component</code> ist das gemeinsame Interface für <code>ConcreteComponent</code> (das Objekt) und <code>Decorator</code> (die Hülle).

Aufgabe 10 (10 Punkte)

Bitte markieren Sie jedes Auswahlfeld, bei dem die links stehende Aussage eine Eigenschaft des darüber stehenden Entwurfsmusters ist. Es können **keines, eines oder mehrere** Felder **pro Zeile** auszuwählen sein.

Aussage	Auswahl (Korrekt)	Erklärung
sehr viele Klassen können entstehen	Factory-Method	Durch die parallelen Vererbungshierarchien (für jedes <code>ConcreteProduct</code> oft ein <code>ConcreteCreator</code>) entstehen viele Klassen.
sehr viele Objekte können entstehen	<i>(Keines)</i>	Dies entsteht eher durch fehlendes <i>Singleton</i> .
sehr viele Methoden können entstehen	Visitor	Im <code>Visitor</code> -Interface muss für jede konkrete Element-Klasse eine eigene <code>visit()</code> -Methode definiert werden. Bei vielen Element-Typen explodiert die Methodenanzahl.
Anzahl erzeugter Objekte kontrollierbar	Singleton	Garantiert genau eine Instanz.
verwandte Operationen zentral verwaltet	Visitor	Das Verhalten (der Algorithmus) wird aus den Klassen extrahiert und zentral im Visitor gebündelt.
häufig als Anti-Pattern angesehen	Singleton	Wegen globalem Status, versteckten Abhängigkeiten und schwerer Testbarkeit.
mehrere Arten <i>primitiver Operationen</i>	Template-Method	GoF definiert explizit Kategorien von Operationen für die Template-Methode (z.B. konkrete, abstrakte/primitive, Hooks).
das Hollywood-Prinzip spielt eine Rolle	Factory-Method, Template-Method	"Don't call us, we'll call you." Die Oberklasse ruft die Implementierung in der Unterklasse auf. (Factory-Method ist oft eine Anwendung dieses Prinzips).
<i>Element</i> ist ein Bestandteil	Visitor	Das Interface <code>Element</code> definiert die <code>accept(Visitor)</code> Methode.
verwendet häufig <i>Hooks</i>	Template-Method, Factory-Method	